LLDD BE - Job 9 SyncNewStoreToDocument

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

บันทึกร้านเปิดใหม่เข้าเอกสาร: อ่านโปรไฟล์ร้านเปิดใหม่และค่า forecast/adjust แล้วบันทึกเข้า sgi_document_new_stores ผ่าน Document Service โดยตรง แทนการเขียนไฟล์ BPM06002O และ SFTP ไป impactprofile

1.2 ขอบเขตงาน

- Main class/script: document.service.syncNewStores / (internal scheduler / service)
- Phase: C
- Output: sgi_document_new_stores (DB)
- Estimate: 10 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	13 ชั่วโมง = implementation 10 + unit test 3 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	`ExportOpenStore` (`fcsJar/src/th/co/gosoft/fgi/main/ExportOpenStore.java`)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success, data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-8-CreateCompensationDocument.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-sync-new-store-to-document` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-sync-new-store-to-document INPUT='{"docNo":"2026/00123"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"docNo":"2026/00123"}' sgi-sync-new-store-to-document`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: ไม่รับ args
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
`docNo`	string	`null` = ทุกเอกสารที่เข้าเงื่อนไข	`YYYY/xxxxx`	ของใหม่
`impactMonth`	string	`null`	`YYYY-MM`	ของใหม่
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- prune เฉพาะแถว `source_system = 'FGI'` — แถวที่คนเพิ่มเอง (USER) ห้ามลบไม่ว่าจะส่ง arg แบบไหน

2.2 Input / Output

Stage	Contract for implementation
Input	New-store compensation rows linked to active impact-process records (writes to sgi_document_new_stores directly; no export file).
Progress	query eligible new-store rows, filter process errors, write outbound new-store payload, insert confirm-receive rows, upload/export, backup, notify.
Output	New-store sync payload/output and confirm-receive rows keyed by NEW_STORE_INFO_ID/month/year.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	query ร้านเปิดใหม่ สถานะ I + forecast + ยังไม่ sync
3	มี sgi_compensation_documents ของ impact_process_id แล้ว? | No: คงสถานะรอ sync / log pending
4	compensate_percent ครบและอยู่ในช่วง 0..100 ทุกแถว? | No: COMPENSATE_PERCENT_INVALID + rollback ก่อน upsert/prune (COALESCE(adjust_compensate_percent, forecast_compensate_percent) ต้องไม่เป็น NULL)
5	upsert sgi_document_new_stores (compensate_percent = COALESCE(adjust_compensate_percent, forecast_compensate_percent) · compensation_amount = COALESCE(adjust_amount, forecast_amount))
6	validate allocation percent รวมต่อ doc_no (ต้องรวมได้ 100 ก่อน submit workflow)
7	insert sgi_interface_transactions: data_name = NEW_STORE · direction = INTERNAL · status = COMPLETED (ไม่สร้างไฟล์ BPM06002O แล้ว — เขียน DB ตรงจึงไม่มี ACK ให้รอ)
8	จบ

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 9 คัดลอกร้านเปิดใหม่เข้าเอกสาร — กติกาเดียวกับ Job 7 เรื่องแถวที่คนคีย์เอง บวกกฎ %ชดเชยรวมต้องเท่ากับ 100%

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
แถวนี้ลบได้หรือไม่ (prune)	`sgi_document_new_stores.source_system`	ลบได้เฉพาะ `source_system = 'FGI'` ที่ไม่มีอยู่ในชุด impact ปัจจุบันแล้ว	✗ แถว `source_system = 'USER'` ห้ามลบทุกกรณี ไม่ว่าจะส่ง argument แบบไหน
%ชดเชยของเอกสารถูกต้องหรือไม่	`sgi_document_new_stores.compensate_percent` (`CHECK BETWEEN 0 AND 100`)	ผลรวมของทุกร้านเปิดใหม่ในเอกสารเดียวกันต้อง = 100% พอดี	ไม่เท่า 100% = ยกเลิกทั้งเอกสารนั้น พร้อม reason ห้าม commit ครึ่งทาง (เอกสารจะคำนวณยอดผิด)
ยอดชดเชยต่อร้านมาจากไหน	`sgi_document_new_stores.compensation_amount` ← `sgi_fgi_impact_stores.forecast_compensation_amount` / `.adjust_compensation_amount`	ใช้ `COALESCE(adjust_compensation_amount, forecast_compensation_amount)` — ค่าที่คนปรับชนะค่าที่ระบบคำนวณเสมอ	ไม่มีทั้งคู่ = 0 (คอลัมน์ `NOT NULL DEFAULT 0`) ไม่ใช่ error

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
`source_system`	`FGI` = ระบบนำเข้าให้ (ลบ/อัปเดตได้) · `USER` = คนคีย์เอง (ห้ามแตะ)	คอลัมน์ `source_system`
ผลรวม %ชดเชย	ต้อง = 100% พอดีต่อ 1 เอกสาร	กติกาธุรกิจ (SDD GI)

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	0 18 7-31 * *	แก้ไขได้	เหลื่อมหลัง Job 8 (17:30) เพราะต้องรอ doc_no · เวลาเหลื่อมไม่ใช่การรับประกัน — ต้องตั้ง dependency ที่ AWS Batch

Field / UI	Format	Validation	Behavior
Target table	sgi_document_new_stores	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	upsert ด้วย doc_no / new_store_code
กฎ Forecast / Percent	COALESCE(adjust_amount, forecast_amount) จาก sgi_fgi_impact_compensations	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ค่า adjust มาก่อน forecast เสมอ; NULL หรือค่านอกช่วง 0..100 ต้อง reject ก่อน upsert
เงื่อนไขเลือกข้อมูล	ร้านเปิดใหม่ สถานะ I + forecast + ยังไม่ sync	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	

2.6 หลักฐานที่ต้องเก็บตอนรัน

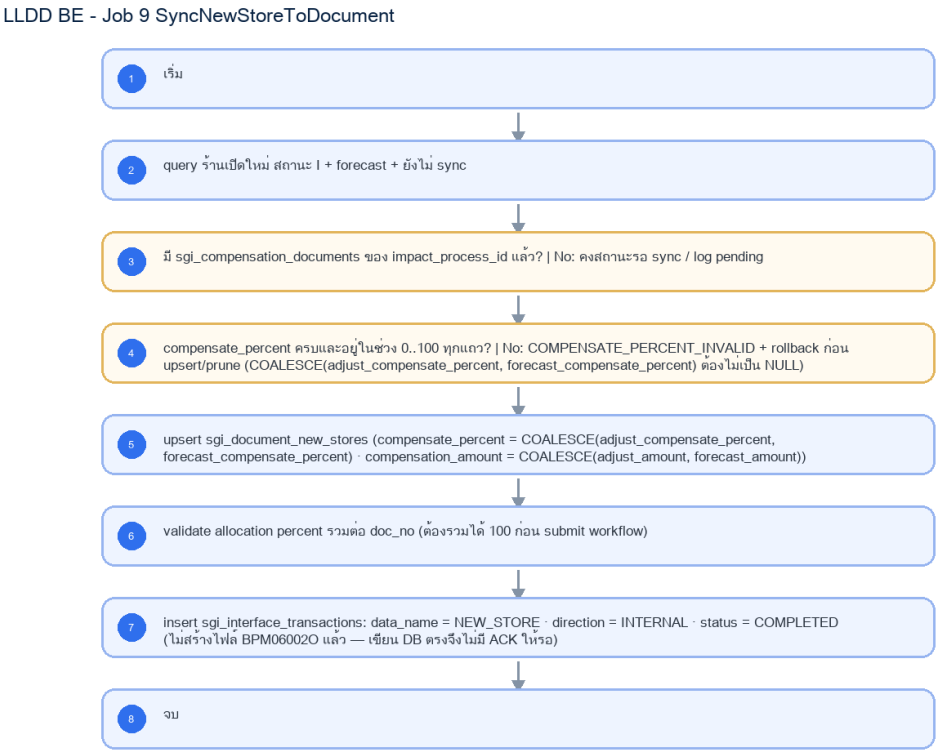
Evidence	Job-specific value	Acceptance
Input identity	New-store compensation rows linked to active impact-process records (writes to sgi_document_new_stores directly; no export file).	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	New-store sync payload/output and confirm-receive rows keyed by NEW_STORE_INFO_ID/month/year.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	UNIQUE(doc_no,new_store_code); upsert + prune เฉพาะ source_system=FGI ให้ target ตรง impact set ปัจจุบัน โดยไม่ลบแถว USER	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	validate source percent ต้องไม่เป็น NULL และอยู่ 0..100 ก่อน upsert; จากนั้น upsert + prune ร้านของ doc_no, validate ผลรวม 100% และ tracking (direction=INTERNAL) ใน transaction เดียว; invalid/ไม่ครบให้ rollback ก่อน prune	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	internal service account least privilege; ไม่มี SFTP/BPM credential หรือ editable external endpoint	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

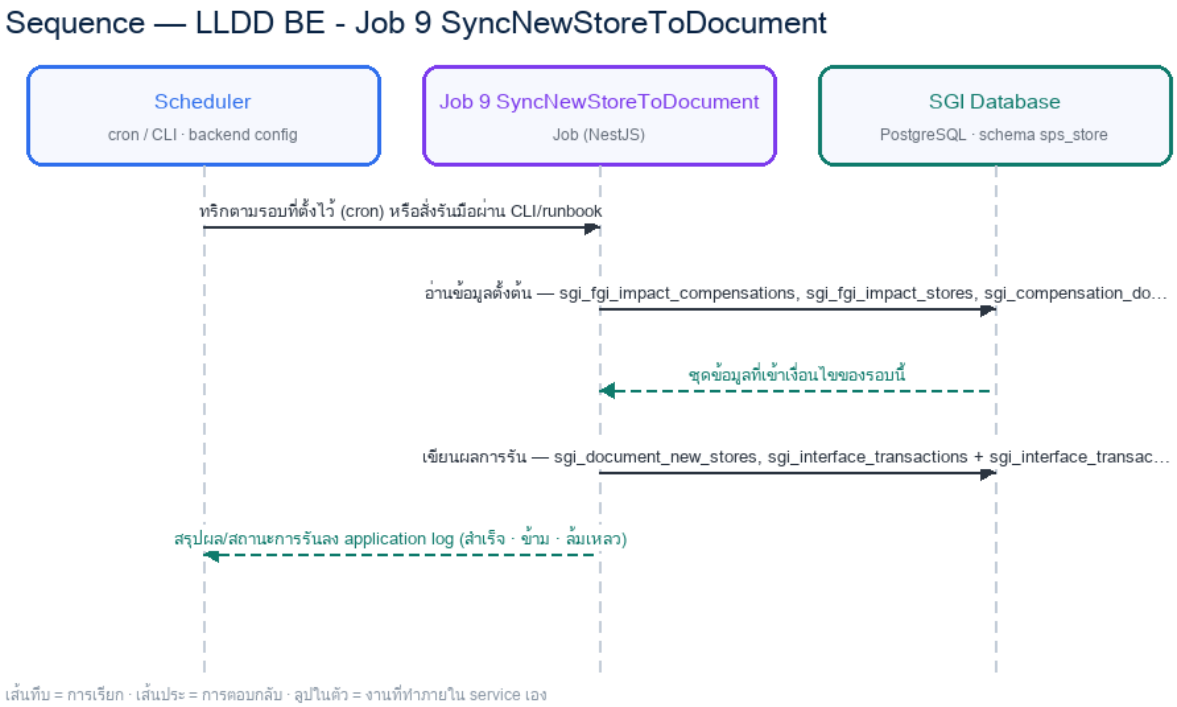
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportOpenStore.java	1-22	Legacy main entrypoint; constant job name is ExportNewStoreToBPM.
fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java	404-516, 893-961	Query new stores, create payload content, upload, backup, notification.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	1558-1594	Query new-store rows eligible for export.

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 9 SyncNewStoreToDocument



รูปที่ 2: Sequence diagram: LLDD BE - Job 9 SyncNewStoreToDocument

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_fgi_impact_compensations	R	ยอด forecast_amount / adjust_amount รายงวด — ที่มาของ %ชดเชย (ตาราง F1)
sgi_fgi_impact_stores	R	โปรไฟล์ร้านเปิดใหม่และค่า forecast/adjust รายงวด
sgi_compensation_documents	R	หา doc_no จาก impact_process_id
sgi_document_new_stores	W	บันทึกร้านเปิดใหม่เข้าเอกสารโดยตรง
sgi_interface_transactions	W	tracking ภายใน: direction=INTERNAL · status=COMPLETED (ไม่มี ACK ให้รอเพราะเขียน DB ตรง)

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-9` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-9-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-9-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-09-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

5 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

การลบแถวเก่า (prune)

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	แถวมาจากระบบนำเข้า	`source_system = 'FGI'` และไม่มีในชุด impact ปัจจุบัน	ลบได้

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-02	• แถวที่คนคีย์เอง	`source_system = 'USER'`	ห้ามลบทุกกรณี

%ชดเชย

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-03	ผลรวมครบ	ร้านเปิดใหม่ 2 แถว 60% + 40%	บันทึกได้
UT-04	• ผลรวมไม่ครบ 100	60% + 30% = 90%	ยกเลิกทั้งเอกสาร ไม่บันทึกครึ่ง ๆ

ยอดชดเชยต่อร้าน

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-05	มีค่าที่คนปรับแล้ว	`adjust_compensation_amount` = 12,000 · `forecast` = 10,000	ใช้ `COALESCE(adjust, forecast)` = 12,000

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportOpenStore.java — lines 1-22

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportOpenStore.java
Original lines	1-22
Responsibility	Legacy main entrypoint; constant job name is ExportNewStoreToBPM.

```
package th.co.gosoft.fgi.main;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;

import th.co.gosoft.fcs.utils.LogUtils;
import th.co.gosoft.fgi.controller.ExportController;

public class ExportOpenStore {

private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
private static ExportController exportController = (ExportController) context.getBean("exportController");
// private static ExportBean config = (ExportBean) context.getBean("fgiExportConfig");

public static void main(String[] args){
LogUtils.info(ExportOpenStore.class,"Start => export NEW_STORE_INFO to BPM");
exportController.exportOpenNewStore();
LogUtils.info(ExportOpenStore.class,"End => export NEW_STORE_INFO to BPM");
}


}
```

J2. ExportController.java — lines 404-415

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	404-415
Responsibility	Query new stores, create payload content, upload, backup, notification.

```
public void exportOpenNewStore(){
  String fileName = "";
Calendar calStart = Calendar.getInstance(Locale.US);
EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
informBean.setSystemCode(FgiConstant.SYSTEM_CODE);
informBean.setJobName(FgiConstant.JOB_NAME_EXPORT_OPEN_STORE);
informBean.setStartDateTime(calStart.getTime());
informBean.setImportFileName("");
try {
configFtp = (FtpBean) context.getBean("fgiExportOpenStore");
setValueConfig();
List<Map> newStoreList = exportService.queryOpenNewStore();
```

J2. ExportController.java — lines 505-516

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	505-516
Responsibility	Query new stores, create payload content, upload, backup, notification.

```
paramTransactionStatus.releaseSavepoint(savepoint);
LogUtils.info(getClass(),"--Transaction is Rollback--");
informBean.setErrorMsg("" + ex);
informBean.setStatus(FgiConstant.JOB_STATUS_FAIL);
return new Boolean("false");
    }
    }});
if(!result){
return "";
}
return fileName;
}
```

J2. ExportController.java — lines 893-904

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	893-904
Responsibility	Query new stores, create payload content, upload, backup, notification.

```
public String appendContentOpenNewStore(Map newStore) throws ParseException{
StringBuilder content = new StringBuilder();
String delimiter = FgiConstant.DELIMITER;
String temp = new String();
if (newStore != null) {

content.append(TextUtils.nullReturnEmpty(newStore.get("NEW_STORE_INFO_ID")) + delimiter);
content.append(TextUtils.nullReturnEmpty(newStore.get("STORECODE_I")) + delimiter);
content.append(TextUtils.nullReturnEmpty(newStore.get("RADIUS")) + delimiter);
content.append(TextUtils.nullReturnEmpty(newStore.get("RADIUS_UNIT")) + delimiter);
content.append(TextUtils.nullReturnEmpty(newStore.get("DISTANCE")) + delimiter);
content.append(TextUtils.nullReturnEmpty(newStore.get("STORECODE_N")) + delimiter);
```

J2. ExportController.java — lines 950-961

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	950-961
Responsibility	Query new stores, create payload content, upload, backup, notification.

```
content.append(delimiter);

content.append(TextUtils.nullReturnEmpty(newStore.get("UPDATE_BY")) + delimiter);

temp = TextUtils.nullReturnEmpty(newStore.get("UPDATE_DATE"));
if(StringUtils.isNotEmpty(temp)){
content.append(th.co.gosoft.fcs.utils.DateUtils.changeformatString(temp, FgiConstant.DATE_FORMAT_FROM_QUERY,
FgiConstant.DATE_FORMAT_DDMMYYYY_TIME, Locale.US, Locale.US));
}
content.append(FgiConstant.NEW_LINE);
}
return content.toString();
}
```

J3. ExportJdbc.java — lines 1558-1569

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1558-1569
Responsibility	Query new-store rows eligible for export.

```
public List<Map> queryOpenNewStore(){
StringBuilder sql = new StringBuilder();
sql.append(" select si.new_store_info_id, si.storecode_n, si.name_n, si.opendate_n, si.closedate_n, ");
sql.append(" si.zone_n, si.branchtype_n, si.juristic_id_n, si.juristic_name_n, ");
sql.append(" fis.radius, fis.radius_unit, fis.distance, fis.distance_unit, ");
sql.append(" fnsc.compensate_month as month, fnsc.compensate_year as year, fnsc.compensate_seq, fnsc.compensate_seq_no,");
sql.append(" nvl(fnsc.compensate_adjust_n,fnsc.compensate_forecast_n) as compensate_forecast_n,
nvl(fnsc.compensate_adjust_percent_n,fnsc.compensate_forecast_percent_n) as compensate_forecast_percent_n, ");
sql.append(" null as compensate_adjust_n, null as compensate_adjust_percent_n, ");
sql.append(" fnsc.create_by, fnsc.create_date, fnsc.update_by, fnsc.update_date, ");
sql.append(" op.storecode_i, si.franchisee_fname_n, si.franchisee_lname_n, op.impact_process_id ");
sql.append(" from fgi_new_store_info si ");
sql.append(" inner join fgi_impact_store_on_process op on op.impact_process_id = si.impact_process_id ");
```

J3. ExportJdbc.java — lines 1583-1594

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1583-1594
Responsibility	Query new-store rows eligible for export.

```
sql.append(" from fgi_confirm_receive_data rd ");
sql.append(" inner join fgi_new_store_info fnsi1 on rd.data_name = '"+FgiConstant.DATA_NEW_STORE+"' ");
sql.append(" and fnsi1.new_store_info_id = rd.transaction_pk ");
sql.append(" and fnsi1.compensate_month = rd.month ");
sql.append(" and fnsi1.compensate_year = rd.year ");
sql.append(" where fnsi1.storecode_i = si.storecode_i ");
sql.append(" and fnsi1.compensate_month = si.compensate_month ");
sql.append(" and fnsi1.compensate_year = si.compensate_year ) ");
List<Map> resultList = jdbcTemplate.queryForList(sql.toString());
LogUtils.info(getClass(),"query FGI_NEW_STORE_SALES for export: "+resultList.size());
return resultList;
}
```